Cameron

Workflow Automation & Reliability — Project-Based

Workflow Repair Desk · workflowrepairdesk@gmail.com
github.com/workflowrepairdesk/n8n-workflow-reliability-proof

PROFILE

Automation builder focused on bounded, testable workflow repairs: clear acceptance criteria, deterministic validation, visible failure behavior, safe sample data, and practical handoff/recovery notes. Available immediately for asynchronous, project-based remote cooperation.

CORE CAPABILITIES

- n8n workflow JSON and JavaScript Code nodes
- Webhooks, structured JSON, validation, scoring, and routing
- Node.js tests and GitHub Actions CI
- Browser, file, script, and messaging automation
- Human-approval boundaries for consequential actions
- Runbooks, rollback gates, verification, and evidence collection

WORKING MODEL

- Start with one measurable workflow outcome
- Agree on an acceptance test before implementation
- Use sanitized data for initial diagnosis
- Keep production credentials in the client environment
- Deliver exportable artifacts and recovery notes
- Quote larger work after a small paid trial

SELECTED TECHNICAL PROOF

n8n lead-triage reliability proof

- Webhook intake → JavaScript Code node → structured response workflow.
- Companion Node implementation with four passing automated tests.
- Three parity fixtures verify that JavaScript embedded in the n8n JSON matches the companion implementation.
- Public GitHub Actions run, safe synthetic fixtures, and explicit validation failures.

OpenClaw on Ubuntu reliability package

- Staged deployment runbook covering dedicated-user setup, systemd recovery, firewall/SSH checks, secrets boundaries, verification, and rollback.
- Read-only host evidence collector designed to avoid credential capture.

TOOLS

n8n · JavaScript · Node.js · Git/GitHub · GitHub Actions · webhooks/REST concepts · OpenClaw · browser automation · Linux/systemd operational planning

Evidence boundary: the portfolio above is self-directed technical proof, not represented as paid client delivery. The n8n tests prove JavaScript/code parity and expected fixtures; they are not claimed as a full n8n import/runtime execution. No paid-client n8n deployments are claimed.